mathematics

MDPI

Article

MelodyDiffusion: Chord-Conditioned Melody Generation Using a Transformer-Based Diffusion Model

Shuyu Li [1] and Yunsick Sung [2,*]

[1] Department of Multimedia Engineering, Graduate School, Dongguk University-Seoul, Seoul 04620, Republic of Korea
[2] Division of AI Software Convergence, Dongguk University-Seoul, Seoul 04620, Republic of Korea
* Correspondence: sung@dongguk.edu; Tel.: +82-2-2260-3338

Abstract: Artificial intelligence, particularly machine learning, has begun to permeate various real-world applications and is continually being explored in automatic music generation. The approaches to music generation can be broadly divided into two categories: rule-based and data-driven methods. Rule-based approaches rely on substantial prior knowledge and may struggle to handle large datasets, whereas data-driven approaches can solve these problems and have become increasingly popular. However, data-driven approaches still face challenges such as the difficulty of considering long-distance dependencies when handling discrete-sequence data and convergence during model training. Although the diffusion model has been introduced as a generative model to solve the convergence problem in generative adversarial networks, it has not yet been applied to discrete-sequence data. This paper proposes a transformer-based diffusion model known as MelodyDiffusion to handle discrete musical data and realize chord-conditioned melody generation. MelodyDiffusion replaces the U-nets used in traditional diffusion models with transformers to consider the long-distance dependencies using attention and parallel mechanisms. Moreover, a transformer-based encoder is designed to extract contextual information from chords as a condition to guide melody generation. MelodyDiffusion can automatically generate diverse melodies based on the provided chords in practical applications. The evaluation experiments, in which Hits@k was used as a metric to evaluate the restored melodies, demonstrate that the large-scale version of MelodyDiffusion achieves an accuracy of 72.41% (k = 1).

Keywords: melody generation; conditional generation; diffusion model; transformer

MSC: 68T99

check for updates

Citation: Li, S.; Sung, Y. MelodyDiffusion: Chord-Conditioned Melody Generation Using a Transformer-Based Diffusion Model. *Mathematics* **2023**, *11*, 1915. https://doi.org/10.3390/math11081915

Academic Editors: Adrian Sergiu Darabant and Diana-Laura Borza

Received: 8 February 2023
Revised: 3 April 2023
Accepted: 17 April 2023
Published: 18 April 2023

Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In recent years, significant advancements have been achieved in the automatic music generation field due to the development of machine learning, with numerous algorithms and techniques having been developed to produce high-quality original music [1,2]. Music generation methods can be broadly classified into rule-based and data-driven approaches.

Initially, owing to a lack of computational recourse and massive data support, rule-based approaches relying on predefined rules and constraints were used to generate music in various styles, such as classical, jazz, and pop music. These approaches often use musical knowledge representation systems to encode musical concepts and rules, such as tonality, rhythm, and harmony [3–6]. However, analyzing and encoding music theory into artificial features using rule-based approaches is difficult. Furthermore, even if rules are defined, it is challenging to apply them to other genres, styles, and instruments.

Data-driven approaches can utilize large amounts of data more efficiently than rule-based methods owing to the large increase in data volume. Data-driven approaches employ deep learning techniques to analyze and generate music based on datasets. Generative

adversarial networks (GANs) have significantly advanced the music generation field. For example, continuous recurrent neural networks with adversarial training (C-RNN-GAN) [7], which are composed of long short-term memory (LSTM)-based generators and bidirectional LSTM-based discriminators, generates classical music by learning from a training dataset. SeqGAN [8] is another GAN-based model that uses reinforcement learning (RL) for sequential generation. It can be trained using data that consist of sequences of discrete tokens. SeqGAN models a generator using a stochastic policy in RL to bypass the generator differentiation problem and perform gradient policy updates directly. SeqGAN is specifically designed for sequence generation and can be trained on various data types. The goal of OR-GAN [9] is to improve the quality of the samples generated by SeqGAN. OR-GAN combines adversarial training with expert-based rewards and RL to generate samples that maintain information learned from data. In this manner, the sample diversity is retained, and the desired metrics are improved. The effectiveness of these approaches has been demonstrated in the generation of molecules encoded as text sequences and musical melodies. However, long-distance sequential data cannot be handled via these approaches owing to the backpropagation process through the time of the LSTM.

In addition to the combination of RNNs and GANs, MidiNet [10] generates music in the symbolic domain using a convolutional neural network (CNN)-based GAN model, allowing it to generate realistic symbolic music. The model incorporates a conditional mechanism to generate original melodies based on a chord sequence or via conditioning based on previously used melody bars. MuseGAN [11] is another GAN model based on CNNs that generates symbolic multitrack music. It includes the jamming, composer, and hybrid models, which are designed to address the unique challenges of music generation, such as its temporal nature and the interdependence of multiple tracks. Inco-GAN [12] is a type of polyphonic music that uses a CNN-based inception model for the conditional generation of polyphonic music that can be freely adjusted in terms of length. In addition, ChordGAN [13] is a chord-conditioned melody generation approach that uses a conditional GAN architecture and appropriate loss functions, similar to image-to-image translation algorithms. It can be used as a tool for musicians to learn compositional techniques for different styles using the same chords and automatically generate music.

Although the use of CNNs has improved the ability of models to extract local features, the issue of gradient vanishing that occurs when processing long-distance sequences has not been addressed. Moreover, balancing the generator and discriminator during dynamic learning is difficult, making it challenging to guarantee the convergence of the GANs during training.

The denoising diffusion probabilistic model (DDPM) [14] is a type of probabilistic generative model that includes the forward and reverse processes. A key benefit of DDPM is that it can be used to denoise data without explicit labels or supervision. This makes it particularly useful for denoising data in unsupervised learning, in which it is difficult or impossible to obtain labeled data. This model can help to address convergence issues in generative models and has achieved promising results in the generation of high-quality images. Moreover, the transformer module, which is used extensively in language modeling, allows the model to focus directly on all input words simultaneously and measure their importance using the attention mechanism. Thus, the model can efficiently capture the long-term dependencies and make more accurate predictions. Transformers also apply parallel processing, making them more efficient and faster to train than RNNs.

This paper proposes a method for chord-conditioned melody generation using a transformer-based diffusion model known as MelodyDiffusion. The following three modifications are made to the traditional diffusion model. First, in the forward process, the inputs of MelodyDiffusion are not continuous image-like data, but they are rather discrete sequences, and no pre-trained variational autoencoder (VAE) is required to map the discrete data to a continuous latent space. Second, in the reverse process, the U-nets of the traditional diffusion model are replaced with transformers, thereby enabling the model to handle discrete sequences and to consider long-distance dependencies. Finally, a

transformer-based encoder is used to embed the conditions for guiding the reverse process, which allows for chord-conditioned music generation.

The main contributions of this paper are summarized as follows: (1) a novel diffusion model that operates directly on discrete data is designed, such that the diffusion model is not limited to image generation. (2) MelodyDiffusion uses transformers instead of U-nets to handle discrete sequences. Furthermore, a transformer-based encoder is developed to realize chord-conditioned melody generation. (3) The experiments reveal that MelodyDiffusion can generate diverse melodies based on the given chords.

2. Related Work

Researchers have started using transformers and diffusion models for music generation. Several representative approaches are described in the following subsections.

2.1. Transformer-Based Music Generation Approaches

As transformers are based on the attention mechanism, they offer more advantages than RNNs in processing sequential data. Some representative studies on the use of transformers for music generation are summarized as follows. CMT [15] is a chord-conditioned melody transformer model for generating K-pop melodies. The model produces the rhythm and pitch separately based on a given chord progression and is trained in two phases. This approach has exhibited promising results for chord-conditioned melody generation. MusicFrameworks [16] is a hierarchical music structure representation and multistep generative process for creating full-length melodies in a manner guided by long-term repetitive structure, chord, melodic contour, and rhythm constraints. The system organizes the full melody into sections and phrases. It generates a melody in each phrase by first generating the rhythm and basic melody using two separate transformer-based networks and then generating the melody in a way which is conditioned on the basic melody, rhythm, and chords. This approach allows for customization and variety by altering the chord, basic melody, and rhythm structures in the music framework. A music generation network based on transformers and guided by music theory has been proposed [17]. This network uses a transformer decoding block to learn the internal information of single-track music and cross-track transformers in order to learn the relationships among tracks involving different musical instruments. A reward network based on music theory is used to optimize network training and produce high-quality music. MELONS [18] is based on a graphical representation of music structure. It uses a multistep generation method with transformer-based networks to factor melody generation into the structure and structure-conditional melody generation. MRBERT [19] uses a pre-trained model that is based on the understanding of melody and rhythm to generate and fill in the melody and chords.

Therefore, in general, transformers have replaced RNNs in music generation tasks and achieved promising results.

2.2. Diffusion Model-Based Music Generation Approcaches

Owing to the remarkable achievements of diffusion models in image generation, researchers have attempted to apply them to waveform data, which are image-like, to accomplish music generation. The text-to-audio system has gained attention for its ability to synthesize general audio based on text descriptions. To address the issues of high computational costs and limited generation quality in previous studies, a diffusion-model-based text-to-audio system called AudioLDM [20] has been proposed. This system can learn continuous audio representations from contrastive language-audio pretraining latent spaces. By using pre-trained models, text embeddings can be used as conditions during sampling while audio mel-spectrogram embeddings are provided to train AudioLDM. Conversely, ERNIE-Music [21] is a diffusion model-based text-to-waveform method that uses lyrics in a textual format as input to control the music generation in waveform format. This is a multimodal generation approach that demonstrates the effective combination of discrete text data and continuous waveforms.

The above two approaches respectively represent music as mel-spectrograms and waveforms, which are continuous image-like data. However, music can also be represented as a discrete sequence composed of notes with specific durations, and transformers can be used to consider the long-distance dependencies between them. This not only extends the training data of diffusion models beyond audio-format music, but also allows for symbolic musical data to be used. A representative approach [22] was proposed for use to train diffusion models on symbolic music data by parameterizing the discrete domain in the continuous latent space of a pre-trained variational autoencoder. This research attempted to use a diffusion model with transformers to generate symbolic music. This enabled the models to generate sequences of latent embeddings through a reverse process and provided parallel generation with a fixed number of refinement steps. Similarly, in the latter work on scalable diffusion models with transformers [23], transformers were used as the main architecture for image generation, replacing U-nets. Furthermore, an autoencoder was used to embed the input into a continuous latent space.

However, the previous approach used a pre-trained biLSTM-based MusicVAE to map a discrete sequence onto a continuous latent space, as well as to decode a discrete sequence from the continuous latent space that is output by the diffusion model. In contrast, the method proposed herein aims to directly embed discrete sequences into the diffusion model, without requiring any intermediate process for conversion into continuous features. Furthermore, a transformer-based encoder is designed to extract contextual information from chords to realize chord-conditioned melody generation.

3. Chord-Conditioned Melody Generation Method

This paper proposes a method for chord-conditioned melody generation using a transformer-based diffusion model known as MelodyDiffusion. We first describe the representations of the melody and chords. Subsequently, we explain the structure and operation of the transformer-based diffusion model.

3.1. Date Representation

The music corpus OpenEWLD [24], which is a dataset consisting of lead sheets in XML format, is used during training. As illustrated in Figure 1, a piano roll is obtained from a lead sheet using the Python library pypianoroll. In the piano roll, each measure is divided into 40 parts. Therefore, each part is a 0.025 measure length; for example, "Duration: 0.25" indicates that a pitch takes up a 0.25 measure length. In a musical instrument digital interface, each pitch is defined with a unique index ranging from 0 to 127, thereby representing 128 types of pitches; for example, the index of "Pitch: C4" is 60. Thus, "Pitch: C4 Duration: 0.25" represents the sequence "60, 60, 60, 60, 0". It is worth noting that the end of this sequence is replaced with 0 given that the "offset" of the pitch must be clearly indicated. Supposing "Pitch: F4 Duration: 0.25" is followed by "Pitch: F4 Duration: 0.5", if the "offset" is not indicated, it will be difficult to distinguish these from "Pitch: F4 Duration: 0.75".

When a melody sequence is generated, the chord sequence can be obtained by simply supplementing the chord index in the corresponding position. A total of 446 chords have been detected and assigned an index in OpenEWLD, with the index 0 indicating "offset".

During training, eight consecutive measures are randomly extracted from any lead sheet in the dataset. The melody and chord sequences, both with a length of 320 (8×40), are obtained using the representation method described above.

3.2. Transformer-Based Diffusion Model

MelodyDiffusion includes forward and reverse processes. As shown in Figure 2, the forward process adds noise to the original melody based on time steps. The reverse process includes a denoising model and a pre-trained encoder, where the denoising model takes the noisy melody generated by the forward process as an input with which to eliminate the original melody. The pre-trained encoder extracts hidden features from the chords and is

connected to the transformers in the denoising model through a cross-attention module to guide the denoising process. The forward and reverse processes are explained in more detail in the following sections.

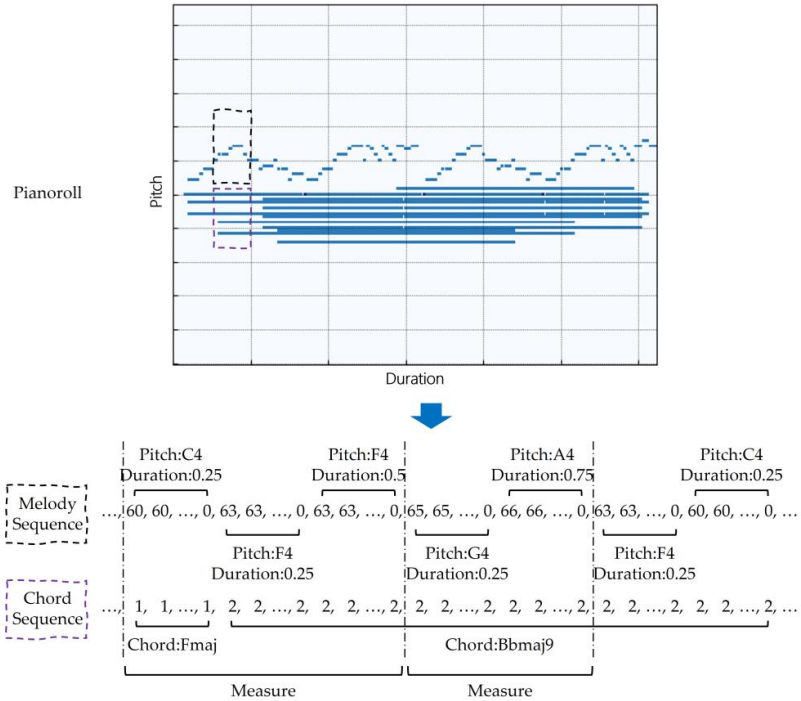

Figure 1. Example of melody and chord sequences converted from a piano roll. In the piano roll, a portion of the corresponding melody and chord were truncated. The black dotted box represents the truncated melody section, while the purple represents the truncated chord. The truncated melody and chord are converted to melody and chord sequences, with each segment between two dotted lines representing the length of one measure.

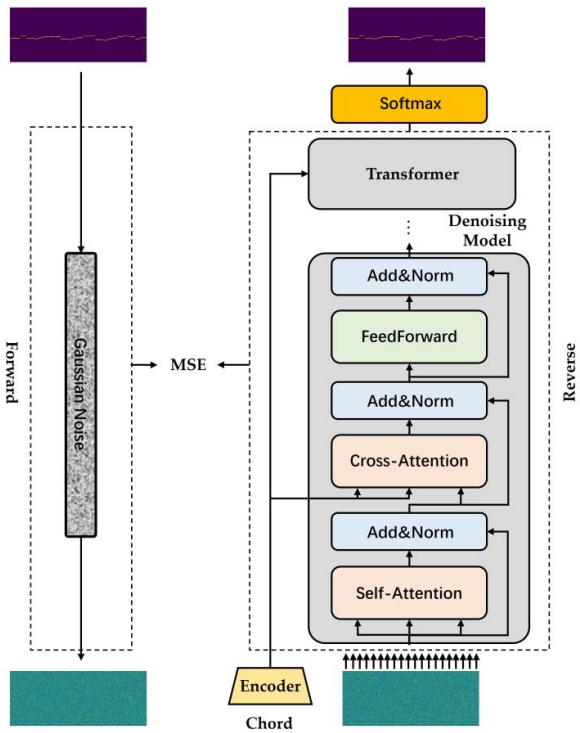

Figure 2. Model structure of MelodyDiffusion.

3.2.1. Notations

This section introduces the notation used in describing the forward and reverse processes, as shown in Table 1. In the forward process, time steps $t \in (1,\ T)$ are used to control adding the noise z on original melody x_0, where the noise z follows a Gaussian distribution. The reason for using Gaussian noise is that the noise must conform to a regular distribution, otherwise the diffusion model is unable predict it. Furthermore, Gaussian noise has been widely used in various diffusion models and has been proven to be effective. In addition, Gaussian noise has a wide range of applications and foundations in statistics and natural sciences; therefore, its use in diffusion models can better simulate real-world noise.

Table 1. Notations and their descriptions.

Notations	Description
t	Time steps
T	Max time step
z	Noise follows a Gaussian distribution
x_0	Original melody
$x_1, x_2, \ldots, x_t, \ldots, x_T$	Melodies with noise added based on time steps $t \in (1,\ T)$
β	Parameter that increases as time step t increases
$\epsilon_\theta(\cdot)$	Denoising model, where [1]θ represents the weights in the model
$\rho_\varphi(\cdot)$	Encoder, where [2]φ represents the weights in the encoder
$c_1,\ c_2,\ \ldots,\ c_n$	Chords that are input as the condition to the encoder
$h_1,\ h_2,\ \ldots,\ h_n$	Hidden features output by the encoder

[1,2] θ and φ represents the weights in $\epsilon_\theta(\cdot)$ and $\rho_\varphi(\cdot)$, and θ is updated during the training of the diffusion model, while φ is frozen.

$x_1, x_2, \ldots, x_t, \ldots, x_T$ represent noisy melodies with noise added based on different time steps t. β varies with t and serves as the parameter for controlling the level of noise added. ϵ_θ and ρ_θ represent the denoising model and encoder, respectively, in the reverse process. $c_1,\ c_2,\ \ldots,\ c_n$ represents the chords that are input as conditions to the encoder. $h_1,\ h_2,\ \ldots,\ h_n$ represents the hidden features output by the encoder.

3.2.2. Forward Process

The forward process follows the time steps 1 to T, recursively adds noise z to the original melody x_0, and saves the noising results of each time step, $x_1, x_2, \ldots, x_t, \ldots, x_T$. Noise is added to the melody at time step t using Equation (1), where β increases as t increases. Therefore, a larger t value indicates that more noise z is added to x_0.

$$x_t = \left(\sqrt{1-\beta_t}\right)x_{t-1} + \sqrt{\beta_t}z_t,\ t \in (1,\ T) \tag{1}$$

By recursively applying Equation (1), the noisy melody x_t at any time step t can be obtained based on the original melody x_0, as shown in Equation (2).

$$x_t = \left(\sqrt{1-\overline{\beta}_t}\right)x_0 + \sqrt{\overline{\beta}_t}z_t,\ t \in (1,\ T) \tag{2}$$

Specifically, each pitch in the melody sequence is embedded using one-hot encoding. As the pitch ranges from 1 to 128, the size of the one-hot encoded vector is 128. Subsequently, each vector undergoes noise processing, whereby the noise follows a Gaussian distribution and is controlled by the parameter β_t. Figure 3 presents the overall process of adding Gaussian noise to the melody sequences. A melody with a length of 320 is embedded into a 320×128 matrix through one-hot encoding. Thereafter, Gaussian noise of the same size as the one-hot encoded melody is generated. Finally, the Gaussian noise is added to the one-hot encoded melody using Equation (2) to obtain the noisy melody.

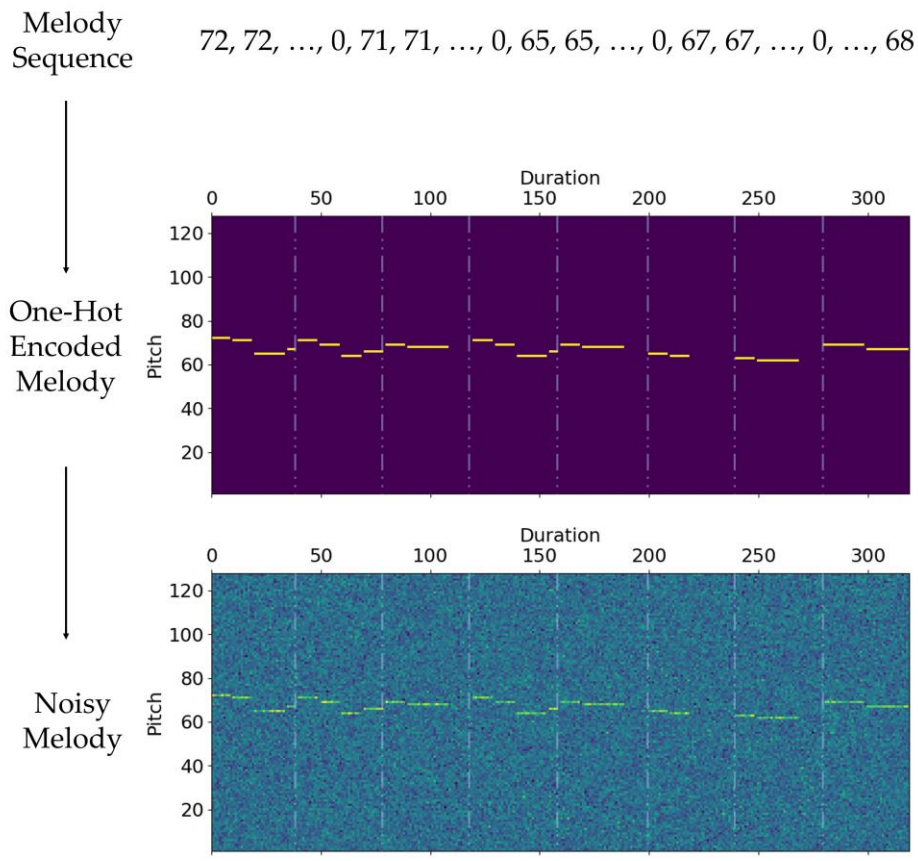

Figure 3. Adding Gaussian noise to the melody sequence.

3.2.3. Reverse Process

In the reverse process, the goal of denoising model ϵ_θ is to infer x_0 recursively from x_t. During the recursive process, Gaussian noise z'_t added to x_{t-1} is predicted based on x_t according to Equation (3). After z'_t is predicted, x'_{t-1} can be obtained easily by using Equation (4), which follows the inverse process of adding noise. Through training, x'_{t-1} gradually approximates the known x_{t-1} from the forward process. $\epsilon_\theta(\cdot)$ represents the denoising model. $\rho_\varphi(\cdot)$ represents the pre-trained encoder, which is composed of transformers using self-attention. It is differentiated from the denoising model $\epsilon_\theta(\cdot)$ given that is a pre-trained model which uses frozen weights during the training of the diffusion model. Moreover, c represents the chord sequence input to the encoder as a conditional input.

$$z'_t \propto \epsilon_\theta\left(\left(\sqrt{1-\overline{\beta}_t}\right)x_0 + \sqrt{\overline{\beta}_t}z_t,\ t\right)\rho_\varphi(c) \tag{3}$$

$$x_{t-1} \approx x'_{t-1} \propto \frac{1}{\sqrt{1-\beta_t}}\left(x_t - \frac{\beta_t}{\sqrt{\overline{\beta}_t}}z'_t\right) \tag{4}$$

Subsequently, the loss is calculated based on z'_t and z_t. In the reverse process, the noise z'_t is predicted based on x_t. In the forward process, the noise z_t added to x_{t-1} is known. The loss for updating the denoising model and encoder is obtained by calculating the difference between z'_t and z_t, and the loss function uses the mean squared error (MSE) as defined in Equation (5).

$$loss = \| z'_t - z_t \|^2 \tag{5}$$

As illustrated in Figure 2, the reverse process comprises two parts: the denoising model and a pre-trained encoder for inputting the chord information. The denoising model is composed of transformer blocks that use both self-attention and cross-attention

mechanisms. The query, key, and value in the self-attention layer originate from the current input. Conversely, in the cross-attention layer, only the query originates from the previous layer, and the key and value are obtained through cross-attention. The structure of the encoder is similar to that of the denoising model, except that all cross-attention layers are replaced with self-attention layers. Its structure is similar to that of BERT [25], and it is pre-trained using the masked language modeling method [25]. The pre-training process is completed in advance. Specifically, chord sequences are randomly masked and fed into the encoder, which learns deeper representations in the process of recovering the masked parts.

As shown in Figure 4, the denoising model is connected to the pre-trained layer through cross-attention. The pre-trained encoder receives the chord sequence $c_1, c_2, \ldots, c_n$ as an input and outputs the hidden features $h_1, h_2, \ldots, h_n$ of the contextual information. Subsequently, these hidden features are fed into each transformer in the denoising model via cross-attention.

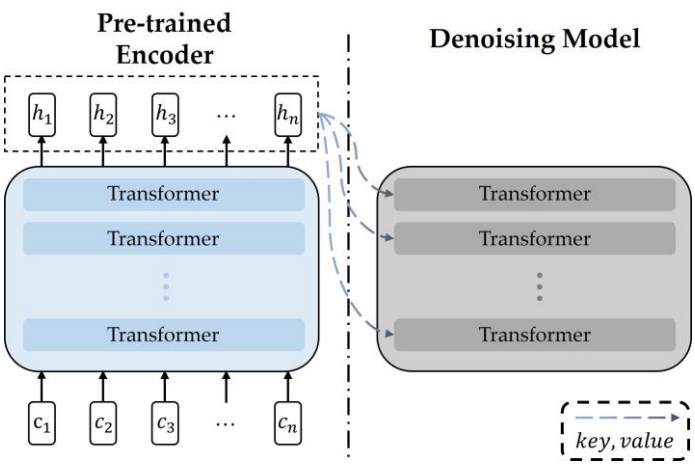

Figure 4. Pipeline of the conditioned-denoising in reverse process.

Algorithm 1 shows the training of MelodyDiffusion, while Algorithm 2 shows the method of sampling new melodies using Gaussian noise and conditional chords after the model has converged.

Algorithm 1 Training

1 : **repeat**
2 : $x_0 = f^{one-hot}(melody\ sequence)$
3 : $c = chord\ sequence$
4 : $t \in Uniform(\{1, \ldots, T\})$
5 : $z_t \sim \mathcal{N}(0, \mathrm{I})$
6 : Take gradient descent step on :
7 : $\nabla_\theta \parallel \epsilon_\theta\left(\left(\sqrt{1-\overline{\beta}_t}\right)x_0 + \sqrt{\overline{\beta}_t}z_t,\ t\right)\rho_\varphi(c) - z_t \parallel^2$
8 : **until** converged

Algorithm 2 Sampling

1 : $x_T \sim \mathcal{N}(0, \mathrm{I})$
2 : **for** t **in** $(1, \ldots, T)$:
3 : $x'_{t-1} = \frac{1}{\sqrt{1-\beta_t}}\left(x_t - \frac{\beta_t}{\sqrt{\overline{\beta}_t}}z'_t\right)$
4 : **return** x'_0

4. Experiments

Hits@k [26] was used as the metric with which to evaluate the quality of the generated melodies. Hits@k is a commonly used evaluation metric for the recommendation models and returns the top k results from the generated list, the latter being sorted by the softmax probability distribution. It represents the average percentage of rankings with a value lower than k in the generated samples. First, MelodyDiffusion-large was designed as a comparison to demonstrate the impact of changes in the hyperparameters of the transformer on the results. Second, "w/o encoder" was used for an ablation experiment. Here, "w/o encoder" represents MelodyDiffusion that did not use an encoder with chord conditions as an input. Finally, the stable diffusion model [27] was reproduced as a baseline, which is a conditional generative model based on U-nets. To adapt discrete data to the stable diffusion model, the input format was set to be slightly different from that of the transformer-based diffusion models.

4.1. Dateset

The Enhanced Wikifonia Leadsheet Dataset (EWLD) is a music lead sheet dataset. OpenEWLD [23] was used in the experiments reported in this paper. It was extracted from EWLD and only contained lead sheets in the public domain. Each lead sheet in OpenEWLD contains the melodies and chords that are required for training. These data were divided into 13,884 sample pairs consisting of eight measures for training and evaluation.

4.2. Experimental Environment

Table 2 lists the hyperparameters that were used during training in the forward process. Number of time steps signified that the maximum value for time step T was 500. β_{start} and β_{end} indicated that the minimum value for β_t in Equations (1) and (2) was 0.0001 and the maximum value was 0.02, with β increasing linearly over time t. The proposed method's models and the comparative models, which consisted of "w/o encoder", and stable diffusion model [27], used the same forward process in the experiment.

Table 2. Hyperparameters used in the forward process.

Hyperparameters	Values
Number of time steps T	500
Schedule of β_t	Linear
β_{start}	0.0001
β_{end}	0.02

Table 3 shows the hyperparameters of the encoder and denoising model in both the base and large versions of the MelodyDiffusion model. The hyperparameter settings of both the base and large versions were based on BERT-base and -large [25], which are types of transformer-based pre-trained model used for representation learning in natural language processing. In this paper, we attempted to use the transformer blocks of BERT as the main architecture of a diffusion model. The encoder and denoising model used the same structure, except that cross-attention was enabled in the denoising model.

Table 4 lists the hyperparameters used by the baseline model. The stable diffusion model was replicated as the baseline. The melody was added as one channel and input into the model in the form of a grayscale image. The model consisted of 12 blocks wherein 6 had functions of down-sampling and 6 had functions of up-sampling. Each block received hidden states extracted from the chords through cross attention from the encoder. The encoder used here was the same as the encoder used in MelodyDiffusion.

Table 3. Hyperparameters of encoder and denoising model in both the base and large versions of MelodyDiffusion.

Hyperparameters	MelodyDiff. (Base)	MelodyDiff. (Large)
Number of layers	12	24
Input size (Length; Embedded dimension)	320×128	320×128
Hidden size	768	1024
FFN inner hidden size	3072	4096
Attention heads	12	16
Attention head size	64	64

Table 4. Hyperparameters of the baseline model.

Hyperparameters	Stable Diffusion
Number of U-net blocks	12
Input size (Channel; Height; Width)	$1 \times 128 \times 320$
Output channel of blocks	128, 128, 256, 256, 512, 512

Table 5 displays the hyperparameters used in the training strategy, including global dropout to prevent overfitting, as well as parameters related to the optimizer AdamW and learning rate. A dropout value of 0.1 was globally used in the reverse process model. The AdamW optimizer was selected to update the model and its learning rate was set to 1×10^{-4}. The parameters relating to the optimizers, namely β_1 and β_2, were set to 0.9 and 0.98, respectively. The warm step, which indicates that the learning rate gradually increases from 0, reached its peak at 500 iterations and then decreased back to 0, following a cosine function.

Table 5. Hyperparameters of the training strategy.

Hyperparameters	Stable Diffusion
Dropout	0.1
AdamW ϵ (Learning rate)	1×10^{-4}
AdamW β_1	0.9
AdamW β_2	0.98
Warm steps	500
Schedule of the learning rate	cosine

4.3. Displays of Training

Figure 5 illustrates the melodies with added noise at different time steps during the forward process. In the original melody x_0, the melody was clearly defined. As the time step t increased to 100, the noisy melody x_{100} became increasingly difficult to recognize. When the time step t increased from 250 to 500, the melody became completely unrecognizable, and the measures were indistinguishable. This demonstrated that noise was gradually added during the forward process. As time steps increased, noise gradually dominated the melody, resulting in a gradual blurring until it became unrecognizable. This further confirms the importance of noise, which provides models with more creativity and diversity, resulting in the generation of more interesting melodies. In the MelodyDiffusion model, by randomly generating Gaussian noise distributions, different styles and qualities of melody generation results can be obtained.

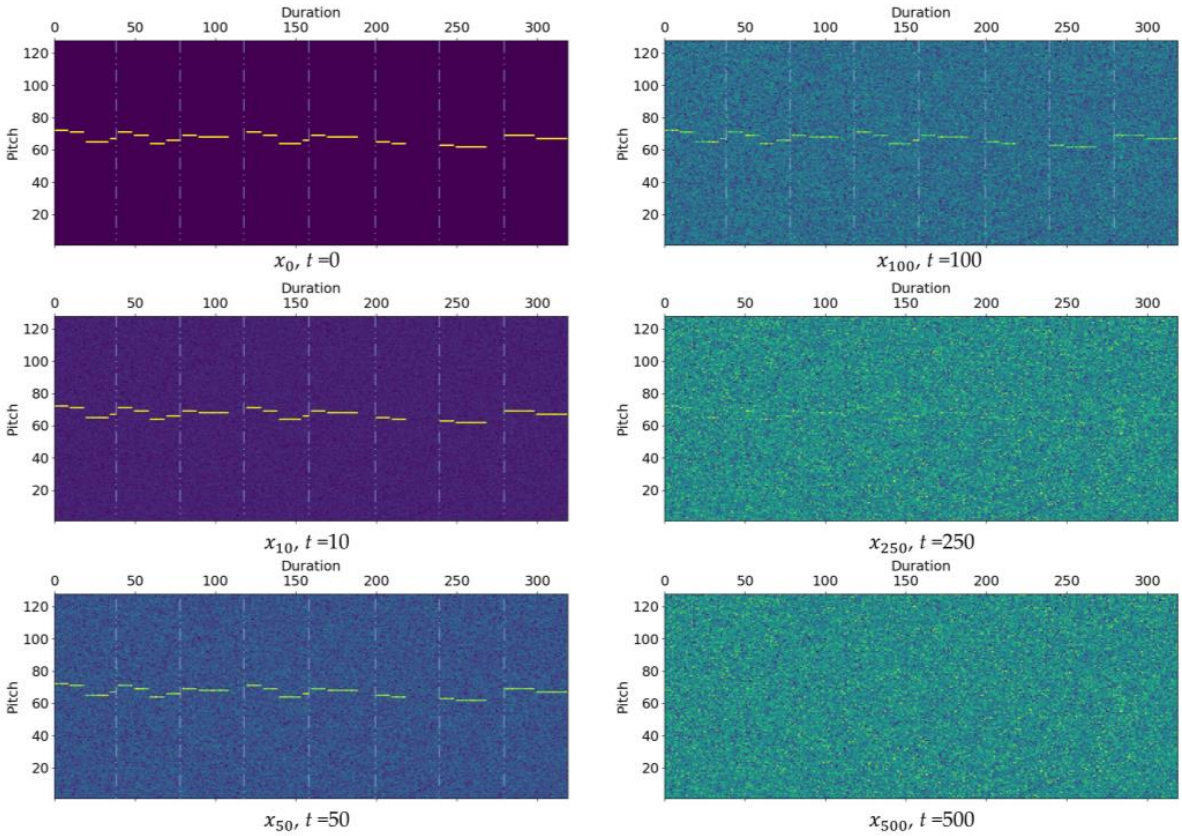

Figure 5. Examples of noise-added melodies at different time steps.

Figure 6 shows the change in MSE loss during the first 250 iterations of training for the MelodyDiffusion base and large models, as well as for the "w/o encoder" and the baseline stable diffusion models during the warm-up phase. Due to the larger number of parameters, the large model had a higher initial loss than the base model. In contrast to the transformer-based models, the stable diffusion model had a low initial loss given that it can predict the noise distribution based on local features by rapidly using convolution. However, as training continued, the transformer models began to excel at predicting noise on discrete data by considering the contextual information in both forward and backward directions. After 100 iterations, the MSE loss between the predicted noise and the actual noise distribution for all four models converged to approximately the same loss.

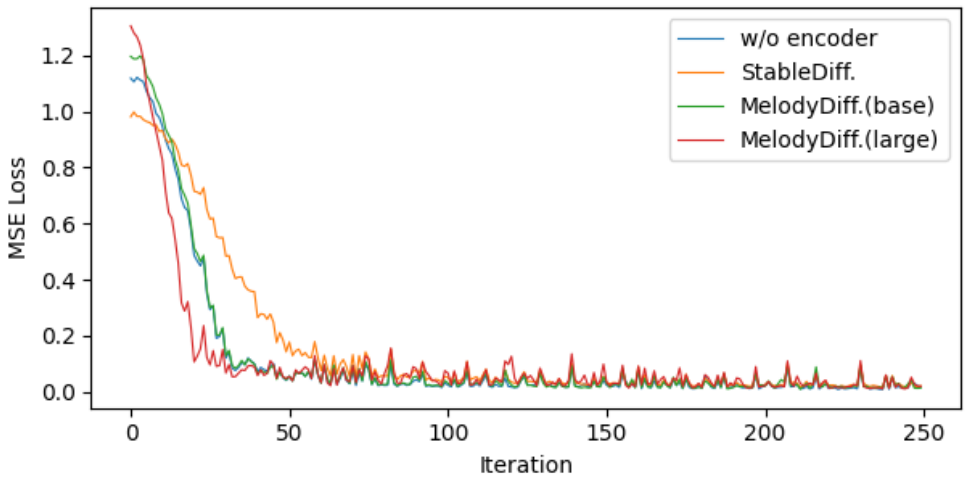

Figure 6. Change in MSE loss during training.

GANs can generate new samples based on noise. However, the goal of GANs is to restore the original sample x_0 from Gaussian noise distribution x_T, which is a challenging task. In contrast, MelodyDiffusion only infers x_{t-1} based on x_t. As shown in Figure 5, it is challenging to directly restore x_0 from x_{500}, making it difficult for the generator of a GAN to update based on the feedback from the discriminator. However, it is relatively easier to predict x_{499}. Furthermore, to make the model converge faster, the training method of the denoising diffusion implicit model (DDIM) [28] was used instead of the traditional DDPM. The reason DDIM was chosen is that the forward process of the diffusion model can include hundreds of steps, and as the DDPM based on Markov chains needs to traverse all time steps to generate a single sample. In DDIM, the forward process can be a non-Markov process and can be accelerated by subsampling.

4.4. Hits@k Evaluation Results

Table 6 presents the evaluation results of the generated melodies using Hist@k. In the restoration process of noisy melodies, which added Gaussian noise at time step T (T = 500) on the basis of the chords given, the reverse process was used to run T recursive denoising operations on the noisy melodies, and the probability distributions of the restored melodies were output through softmax. For Hist@k, k was set to 1, 3, 5, 10, and 20, and the average percentage of the pitch in the original melodies among the top k pitches in the softmax probability distribution was calculated. When k = 1, only the pitch with the highest probability in the softmax probability distribution was considered to evaluate the performance of the restoration, which was the key metric in this. The performance of the base version of MelodyDiffusion was slightly lower than that of the baseline stable diffusion model, being lower by 1.43%; meanwhile, the large version with a similar number of parameters had a performance which was 0.63% higher. However, the performance of the w/o encoder model, which did not use chords as auxiliaries, was worse than that of the other three conditional models. When k > 3, the Hist@k score of the stable diffusion model was higher than that of the MelodyDiffusion models. After analyzing the generated results, it was apparent that the section of the melody with significant changes in pitch affected the transformer's judgment in the restoration process given that it tended to make predictions based on contextual features. In contrast, the stable diffusion model, which utilized a CNN, could restore this section based on local features.

Table 6. Evaluation results of the generated melodies obtained using Hist@k.

	HITS@1 (%)	HITS@3 (%)	HITS@5 (%)	HITS@10 (%)	HITS@20 (%)
MelodyDiff. (base)	70.35	83.13	87.03	90.52	92.95
MelodyDiff. (large)	72.41	84.40	87.12	90.49	92.88
w/o encoder	66.78	80.99	86.98	89.66	91.87
StableDiff.	71.78	82.97	88.45	91.72	93.01

4.5. Comparison of Generated and Real Melodies

Figure 7 illustrates the generated melodies of the original melody (A) and those generated by the base (B) and large (C) versions of MelodyDiffusion, the stable diffusion model (D), and the w/o encoder (E). These melodies were generated by the models based on using random Gaussian noise as an input and utilizing the same chords as the condition (except for w/o encoder). The analysis of the results showed that all models could recognize the measures (with a duration of 40 for each measure). Second, there were obvious noise patterns in the background of (D); these occurred since the stable diffusion model, using CNN, could not eliminate interference based on contextual features. Finally, the three generated melodies in (E) were extremely similar, particularly in the case of the first two measures. This also indicated that the chords could provide diversity to the generated melodies.

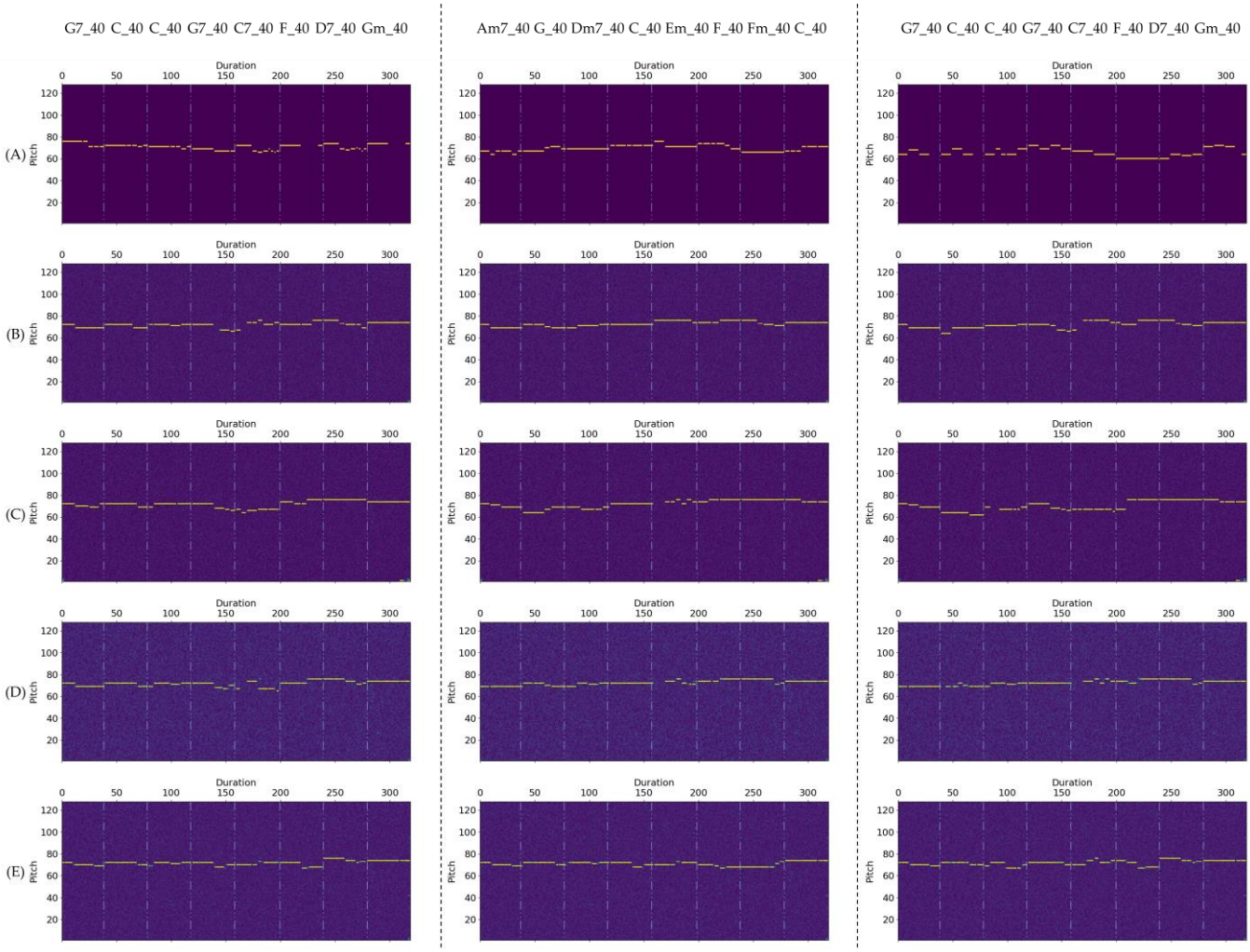

Figure 7. Visual comparison of the Softmax probability distribution of original and generated melodies. (**A**) Original melodies; (**B**) generated melodies based on the base version of MelodyDiffusion; (**C**) generated melodies based on the large version of MelodyDiffusion; (**D**) generated melodies based on the stable diffusion model; (**E**) generated melodies based on the w/o encoder system.

5. Conclusions

This paper proposed a chord-conditioned melody generation method using MelodyDiffusion, which is a diffusion model based on transformers. MelodyDiffusion consists of forward and reverse processes. In the forward process, Gaussian noise is added to melodies following the time step. In the reverse process, the noise is reduced through the denoising model, using the given chords as conditions. Following training, this model could effectively predict the noise distribution from noisy melodies. Hits@k was used as a metric to evaluate the generated melodies. The experimental results showed that the performance of the proposed method reached 70.35% and 72.41% (k = 1) for the base and large versions, respectively, in restoring noisy melodies, with the large version outperforming the baseline model (stable diffusion) by 0.63%. Additionally, the visual comparison showed that MelodyDiffusion could generate more diverse melodies under the condition of given chords compared to the unconditional diffusion model.

Currently, MelodyDiffusion can only generate monophonic melodies based on chords. Considering the practicality of polyphonic music composed of multiple instrument combinations, future research directions include identifying chords and melodies from polyphonic music, which mainly comprises multi-track polyphonic music, without distinguishing chords and melodies compared to datasets with paired chords and melodies.

Moreover, it is necessary to explore the application of MelodyDiffusion to other forms of musical data and generate polyphonic music containing multiple instruments based on these considerations are necessary.

Moreover, the diffusion model was originally proposed and used in the field of image generation; thus, although its main structure was replaced with transformers to handle discrete data with temporal properties in MelodyDiffusion, a limitation on the length of the generated samples exists. While increasing the processing length of transformers can generate longer samples, it also leads to higher training costs. A feasible approach is to guide the generation of subsequent melodies by treating previous melody patterns as conditional inputs, similar to what occurs in the conditioning of chord inputs.

Supplementary Materials: The following supporting information can be downloaded at: https://www.mdpi.com/article/10.3390/math11081915/s1, Demo S1: MIDI files generated by the MelodyDiffusion and MP3 files converted from them.

Author Contributions: Conceptualization, S.L. and Y.S.; methodology, S.L. and Y.S.; software, S.L. and Y.S.; validation, S.L. and Y.S.; writing-original draft, S.L.; writing—review & editing, Y.S. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Ministry of Education of the Republic of Korea and the National Research Foundation of Korea (NRF-2021R1F1A1063466).

Data Availability Statement: Restrictions apply to the availability of these data. Data was obtained from https://github.com/00sapo/OpenEWLD and are available accessed on 1 October 2022. MIDI files generated by the MelodyDiffusion and MP3 files converted from them are available as Supplementary Materials.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Huang, C.A.; Vaswani, A.; Uszkoreit, J.; Shazeer, N.; Simon, I.; Hawthorne, C.; Dai, A.M.; Hoffman, M.D.; Dinculescu, M.; Eck, D. Music transformer: Generating Music with Long-term Structure. *arXiv* **2018**, arXiv:1809.04281.
2. Hawthorne, C.; Stasyuk, A.; Roberts, A.; Simon, I.; Huang, C.A.; Dieleman, S.; Elsen, E.; Engel, J.; Eck, D. Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset. *arXiv* **2018**, arXiv:1810.12247.
3. Salas, H.A.G.; Gelbukh, A.; Calvo, H.; Soria, F.G. Automatic Music Composition with Simple Probabilistic Generative Grammars. *Polibits* **2011**, *44*, 59–65. [CrossRef]
4. Alvaro, J.L.; Miranda, E.R.; Barros, B. Music Knowledge Analysis: Towards an Efficient Representation for Composition. In Proceedings of the 11th Conference of the Spanish Association for Artificial Intelligence (CAEPIA), Santiago de Compostela, Spain, 16–18 November 2005; pp. 331–341.
5. Akama, T. Controlling Symbolic Music Generation based on Concept Learning from Domain Knowledge. In Proceedings of the International Conference on Music Information Retrieval (ISMIR), Delft, The Netherlands, 4–8 November 2019; pp. 816–823.
6. Dubnov, S.; Assayag, G.; Lartillot, O.; Bejerano, G. Using Machine-Learning Methods for Musical Style Modeling. *Computer* **2003**, *36*, 73–80. [CrossRef]
7. Mogren, O. C-RNN-GAN: Continuous Recurrent Neural Networks with Adversarial Training. *arXiv* **2016**, arXiv:1611.09904.
8. Yu, L.; Zhang, W.; Wang, J.; Yu, Y. SeqGAN: Sequence Generative Adversarial Nets with Policy Gradient. In Proceedings of the 31st AAAI Conference on Artificial Intelligence, San Francisco, CA, USA, 4–9 February 2017; pp. 2852–2858.
9. Guimaraes, G.L.; Sanchez-Lengeling, B.; Outeiral, C.; Farias, P.L.C.; Aspuru-Guzik, A. Objective-Reinforced Generative Adversarial Networks (ORGAN) for Sequence Generation Models. *arXiv* **2017**, arXiv:1705.10843.
10. Yang, L.C.; Chou, S.Y.; Yang, Y.H. MidiNet: A Convolutional Generative Adversarial Network for Symbolic-domain Music Generation. *arXiv* **2017**, arXiv:1703.10847.
11. Dong, H.W.; Hsiao, W.Y.; Yang, L.C.; Yang, Y.H. MuseGan: Multi-Track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment. In Proceedings of the 32nd AAAI Conference on Artificial Intelligence, New Orleans, LA, USA, 2–7 February 2018; pp. 34–41.
12. Li, S.; Sung, Y. INCO-GAN: Variable-length music generation method based on inception model-based conditional GAN. *Mathematics* **2021**, *9*, 387. [CrossRef]
13. Lu, C.; Dubnov, S. Chordgan: Symbolic Music Style Transfer with Chroma Feature Extraction. In Proceedings of the 2nd Conference on AI Music Creativity (AIMC), Online, 18–22 July 2021.
14. Ho, J.; Jain, A.; Abbeel, P. Denoising Diffusion Probabilistic Models. *Adv. Neural Inf. Proc. Syst.* **2020**, *33*, 6840–6851.
15. Choi, K.; Park, J.; Heo, W.; Jeon, S.; Park, J. Chord Conditioned Melody Generation with Transformer Based Decoders. *IEEE Access* **2021**, *9*, 42071–42080. [CrossRef]

16. Dai, S.; Jin, Z.; Gomes, C.; Dannenberg, R.B. Controllable Deep Melody Generation Via Hierarchical Music Structure Representation. *arXiv* **2021**, arXiv:2109.00663.
17. Jin, C.; Wang, T.; Li, X.; Tie, C.J.J.; Tie, Y.; Liu, S.; Yan, M.; Li, Y.; Wang, J.; Huang, S. A Transformer Generative Adversarial Network for Multi-Track Music Generation. *CAAI Trans. Intell. Technol.* **2022**, *7*, 369–380. [CrossRef]
18. Zou, Y.; Zou, P.; Zhao, Y.; Zhang, K.; Zhang, R.; Wang, X. MELONS: Generating Melody with Long-Term Structure Using Transformers and Structure Graph. In Proceedings of the International Conference on Acoustics, Speech and Signal Processing (ICASSP), Singapore, 23–27 May 2022; pp. 191–195.
19. Li, S.; Sung, Y. MRBERT: Pre-Training of Melody and Rhythm for Automatic Music Generation. *Mathematics* **2023**, *11*, 798. [CrossRef]
20. Liu, H.; Chen, Z.; Yuan, Y.; Mei, X.; Liu, X.; Mandic, D.; Wang, W.; Plumbley, M.D. AudioLDM: Text-to-Audio Generation with Latent Diffusion Models. *arXiv* **2023**, arXiv:2301.12503.
21. Zhu, P.; Pang, C.; Wang, S.; Chai, Y.; Sun, Y.; Tian, H.; Wu, H. ERNIE-Music: Text-to-Waveform Music Generation with Diffusion Models. *arXiv* **2023**, arXiv:2302.04456.
22. Mittal, G.; Engel, J.; Hawthorne, C.; Simon, I. Symbolic Music Generation with Diffusion Models. *arXiv* **2021**, arXiv:2103.16091.
23. Peebles, W.; Xie, S. Scalable Diffusion Models with Transformers. *arXiv* **2022**, arXiv:2212.09748.
24. Simonetta, F.; Carnovalini, F.; Orio, N.; Rodà, A. Symbolic Music Similarity through a Graph-Based Representation. In Proceedings of the Audio Mostly on Sound in Immersion and Emotion, North Wales, UK, 12–14 September 2018; pp. 1–7.
25. Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. BERT: Pretraining of Deep Bidirectional Transformers for Language Understanding. *arXiv* **2018**, arXiv:1810.04805.
26. Zeng, M.; Tan, X.; Wang, R.; Ju, Z.; Qin, T.; Liu, T.Y. MusicBERT: Symbolic Music Understanding with Large-Scale Pre-Training. In Proceedings of the Findings of the Associations for Computational Linguistics: ACL-IJCNLP, Online, 1–6 August 2021; pp. 791–800.
27. Rombach, R.; Blattmann, A.; Lorenz, D.; Esser, P.; Ommer, B. High-Resolution Image Synthesis with Latent Diffusion Models. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), New Orleans, LA, USA, 19–24 June 2022.
28. Song, J.; Meng, C.; Ermon, S. Denoising Diffusion Implicit Models. *arXiv* **2020**, arXiv:2010.02502.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.